**FINAL DEVELOPER TEST / 01**

# Secure staff access portal

**Candidate brief | Two working days | Approximately 14-16 hours**

Build and demonstrate an isolated prototype of an internal staff access portal. Staff sign in using their own email-based identity, see only their assigned application icons, and open the actual permitted services without retyping service passwords wherever a supported, approved sign-in flow allows it.

## Assessment scenario

This is a greenfield developer test. There is no existing portal or saved-password system for you to inspect, reuse, or migrate. Build a new, isolated assessment portal with synthetic data and dedicated test identities. The task is an internal application access prototype, not a sales CRM or a production deployment.

The required application inventory is ServiceM8, RingCentral, company email, and website forms. The email provider, form platform, account plans, identity provider, and proposed launch flow must be confirmed. Opening a public form is different from accessing its restricted management area.

## The result we want

• One staff identity, a clear dashboard, admin-controlled application assignment, and a supported path into each permitted service using the correct vendor account.

• Actual vendor-side permissions must match each staff member’s role. Hiding an icon does not restrict an account inside the vendor’s service.

• A polished, responsive online demonstration, source code or editable project, test evidence, and a concise implementation, security, and cost plan.

## Assessment boundary

Two days is a prototype and feasibility assessment, not a secure production guarantee or a commitment to integrate all four services regardless of vendor restrictions. Prove at least one real permitted integration when the approved prerequisites are supplied, plus a controlled demo application. Clearly separate live, simulated, blocked, and untested work.

The earlier SafeCertPro Google Cloud cost research remains a separate assignment for both candidates. It is not included in these 14-16 hours. This brief does not replace or alter the earlier tests.

# Confirm feasibility before building

## First checkpoint within the first two hours

Before building integrations, submit the application inventory and proposed scope. Research independently using current official documentation and dated evidence.

• For each of the four applications, record the exact service and target page, account owner, plan, available login methods, MFA requirements, staff account model, and supported SSO or enterprise credential-launcher options.

• For each proposed one-click flow, distinguish supported SSO, an approved enterprise launcher, and simple reuse of an already-authenticated browser session. Identify what must be configured and what the flow actually proves.

• List dependencies: dedicated test accounts, approved test identities, sandbox or test tenant, permitted redirect addresses, identity-provider access, and any vendor or launcher configuration needed.

• State the expected first-login and repeat-login experience, any consent or MFA interaction, cost or plan prerequisites, revocation behavior, and unsupported scenarios.

## Proceed only within approved scope

Build from scratch in an isolated test environment with synthetic data and dedicated test identities. Do not request the owner’s production credentials or access any existing CRM. No existing-system access or migration work is required. Use only explicitly approved test accounts for real vendor demonstrations.

Obtain the owner’s approval before creating persistent credentials or access grants, changing account security settings, enabling paid services, accepting new paid commitments, or using an enterprise credential launcher. Never assume free hosting means vendor integrations are free. Do not require a paid integration merely to take the test.

If a requested flow is unsupported or prerequisites are unavailable, report the exact blocker, evidence, options, cost implications, and a recommended next step. Ask the owner before substituting a service or approach. Continue independent portal work without pretending the blocked integration works.

## Authentication distinctions to verify

ServiceM8 currently offers Google sign-in on its official login page (checked 20 October 2026). That alone does not prove a custom portal can sign every staff member into ServiceM8. Google sign-in and Google Authenticator MFA are different; confirm which the business means. An OAuth API grant is not evidence of a browser session in the service website.

ServiceM8 client deep links can require login (checked 20 October 2026). Prove password-free access with a test account in the browser, not just a link or API call.

# Staff and admin experience

## Individual staff access

• Provide individual email-based staff sign-in using a suitable established authentication mechanism. Only identities explicitly allowed by an administrator may access the portal; possession of a company-looking email address is not enough.

• Show the signed-in person clearly. Display only their assigned applications with recognizable icons, names, useful descriptions, and honest availability states. Never expose a shared administrator account to staff.

• Make launch behavior clear: opening a new tab, completing a first-time sign-in, encountering MFA, or needing an administrator. Include useful errors and a safe way to retry without leaking secrets.

## Admin controls and backend enforcement

• Provide a separate admin area to allow or disable staff identities, assign or revoke applications, and review access activity. Staff must not be able to grant themselves admin or application access.

• Enforce role and application checks on the backend for every protected page, API, and launch request. Direct URLs, modified requests, or another user’s identifiers must not bypass authorization.

• Recheck permissions on subsequent requests after an admin change. Define and demonstrate how disabling a staff member invalidates their portal sessions and prevents further portal access.

## Required advanced feature

**Time-limited application access.** Let an admin grant a staff member access to one application until a specified expiry time. Show the expiry clearly, enforce it on the backend, and record the grant, expiry, and denied post-expiry launch in the audit trail. A browser-only countdown or hidden tile does not satisfy this requirement.

Expiry stops new access through the portal. It does not automatically end an existing vendor session. Demonstrate the vendor’s actual revocation behavior if available, or clearly document the manual step and remaining exposure.

## Interface quality

Deliver a consistent, professional desktop and mobile layout with readable typography, spacing, contrast, keyboard-accessible controls, visible focus, and clear loading, empty, denied, and error states. Avoid unfinished placeholders and buttons that imply functionality they do not have.

Use any appropriate stack. Explain your choices and trade-offs; no particular framework, identity product, or paid integration is prescribed.

# Security boundaries

## Use supported authentication paths

Prefer vendor-supported SSO or a specifically approved enterprise credential launcher when suitable. Do not build a bespoke password vault, scrape saved passwords, inject passwords into vendor login pages, share session cookies, bypass MFA, or use a shared admin login as a shortcut.

If using a managed launcher, explain who can view or recover credentials, how staff identity and vendor permissions are mapped, and how access is revoked. Do not promise that users can never discover a shared credential unless the chosen product and configuration demonstrably enforce that claim.

## Protect the portal

• Use HTTPS, secure session handling, appropriate session expiry and logout, server-side authorization, and standard protection against relevant request forgery, injection, and cross-site scripting risks.

• Keep secrets out of source control, frontend code, URLs, browser storage, screenshots, logs, and submitted documents. Use suitable managed secret handling for any approved server-side secrets.

• Log sign-in outcomes, grants, revocations, admin actions, launches, denials, and expiry events with actor, target, timestamp, and outcome. Do not log passwords, tokens, MFA codes, or full sensitive payloads. Do not claim vendor activity is audited unless it is actually captured.

• Protect the publicly reachable demo with authentication. Use dummy records only, restrict admin access, and document demo cleanup and account removal.

## Prove the integration honestly

Demonstrate at least one actual target service with a dedicated, permitted test account when prerequisites are available. Include a controlled demo application to prove role and access enforcement independently of vendor constraints. Label it as a demo, never as ServiceM8, RingCentral, email, or the forms provider.

For the real integration, show first use and repeat use, the authenticated vendor identity, correct destination, MFA or consent behavior, and the permissions available inside the vendor. An existing browser session alone does not prove the portal established sign-in.

## Revocation has two separate layers

Test portal access revocation and vendor session revocation separately. Establish what happens to a vendor tab opened before access was revoked or expired. Verify a supported vendor session or account revocation mechanism where available; otherwise state the manual administrator action and limitation. Do not claim a full session kill without evidence.

# Work plan and submission

## Day one about seven to eight hours

**Hours 1-2:** Confirm the test scenario, inventory the four intended applications, research supported login flows, identify blockers, and present the feasibility checkpoint and approvals needed.

**Hours 3-5:** Build individual staff sign-in, the admin allowlist, backend roles and application assignments, and the responsive dashboard. Establish the isolated demo and test identities.

**Hours 6-8:** Complete the controlled demo application, implement audit events, and start the approved real integration. Provide a short end-of-day update showing working features, evidence, blockers, and the remaining scope.

## Day two about seven to eight hours

**Hours 9-11:** Complete the permitted real integration where prerequisites allow, admin revocation, session controls, and the time-limited grant feature. Clearly mark blocked or untested applications.

**Hours 12-14:** Run the acceptance tests, fix authorization and usability defects, inspect mobile behavior, and verify the online deployment in a clean browser context.

**Hours 15-16:** Finish evidence and documentation, prepare the short plan, and demonstrate the work. Show one small requested live change and explain its effect. Report actual time spent and unfinished items.

## Submit these deliverables

• **Online demo:** a working HTTPS URL on a free hosting option, not localhost only. State availability limits and expiry. Arrange admin and two staff test identities securely; do not publish reusable credentials in the submission.

• **Editable implementation:** complete source or editable project, dependency information, configuration examples without secrets, and reproducible setup and deployment instructions.

• **README and evidence:** architecture and data flow, access model, integration inventory, test results, dated screenshots or a short recording with secrets redacted, known limitations, and cleanup instructions.

• **Your own short PDF or PowerPoint plan:** proposed full implementation, identity and security approach, rollout steps, unresolved dependencies, and itemized hosting, identity, vendor, and launcher costs. State currency, assumptions, free-tier limits, and one-time versus recurring costs. Verify current prices rather than guessing.

Keep the submission concise and make your own technical decisions. Be ready to explain the code, security boundaries, failed approaches, and cost trade-offs without relying on a scripted demonstration.

# Acceptance tests and completion

Use one admin and two staff identities with different application assignments. Record expected result, actual result, and evidence for every test. Mark blocked or untested results explicitly rather than passing them.

## Required demonstration tests

• **Identity:** an allowed staff member signs in; an unknown or disabled identity is denied; staff cannot access admin actions.

• **Assignment:** Staff A and Staff B see different permitted applications. Direct portal URLs, protected APIs, changed identifiers, and unauthorized launch requests are denied by the backend.

• **Real launch:** the approved vendor integration opens the actual service under the intended test identity. Show first and repeat access, prompts, MFA, and vendor-side role restrictions.

• **Revocation:** revoke an assignment and disable a staff member. Show subsequent portal requests are denied and explain what happens to already-open vendor sessions.

• **Expiry:** grant short-lived access, demonstrate access before expiry and denial after expiry, and show the matching audit records.

• **Session and data safety:** demonstrate logout and expiry behavior; confirm no credentials or tokens appear in client assets, repository, URLs, logs, or submitted evidence.

• **Quality:** verify mobile and desktop layouts, keyboard use, useful error states, deployment access from another browser, and the requested live change.

## Assessment completion and future scope

**Assessment completion:** a working isolated portal and controlled app, the required security and admin behaviors, evidence for one approved real integration when prerequisites are supplied, and a truthful four-application feasibility report. If prerequisites are not supplied, submit the working independent scope and blocker evidence; the real-integration requirement remains pending until the owner agrees a revised scope.

**Future rollout criteria, outside this test:** all four actual services work through approved flows for the required staff roles; vendor account permissions and offboarding are verified; costs and dependencies are approved; and production security review, operational ownership, and support arrangements are complete. These are future criteria only; a production rollout is not assigned by this test.

**Critical failures:** unauthorized access, exposed secrets, MFA bypass, or presenting a mocked integration as live. An evidence-backed vendor limitation is a legitimate finding, not proof of poor ability, but it is not completion of the full rollout.